

CSCI 240 PA7 Submission

Exercise 1

Source code below:

```
package PA7;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class PA7_Ex1 {
    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");
        System.out.println("=== Part A: Manual HashMap Operations ===");
        runManualTests();
        System.out.println("\n=== Part B: Inserting small1k.txt ===");
        timeInsertion("PAs/PA7/Data/small1k.txt", 1000);
        System.out.println("\n=== Part C: Inserting large100k.txt ===");
        timeInsertion("PAs/PA7/Data/large100k.txt", 100000);
    }

    private static void runManualTests() {
        HashMap<Integer, String> map = new HashMap<>();

        int[] keys = {13, 21, 5, 37, 15};
        System.out.print("Inserting keys: ");
        for (int key : keys) {
            System.out.print(key + " ");
            String value = new
StringBuilder(String.valueOf(key)).reverse().toString();
            map.put(key, value);
        }
        System.out.println();

        searchAndPrint(map, 10);
        searchAndPrint(map, 21);

        System.out.println("\n> Removing 20 and 37...");
        map.remove(20);
        map.remove(37);

        searchAndPrint(map, 37);

        System.out.println("\n> Map Entries via Iterator:");
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println("Key: " + entry.getKey() + " -> Value: " +
entry.getValue());
        }
    }

    private static void searchAndPrint(HashMap<Integer, String> map, int
key) {
        if (map.containsKey(key)) {
            System.out.println("Searched for " + key + " -> Found! Value:
" + map.get(key));
        } else {
```

```

        System.out.println("Searched for " + key + " -> Not Found.");
    }
}

private static void timeInsertion(String filePath, int count) {
    int capacity = (int) Math.ceil(count / 0.75);
    HashMap<Integer, String> map = new HashMap<>(capacity);

    long startTime = System.currentTimeMillis();

    try (Scanner scanner = new Scanner(new File(filePath))) {
        int i = 0;
        while (scanner.hasNextInt() && i < count) {
            int key = scanner.nextInt();
            String value = new
StringBuilder(String.valueOf(key)).reverse().toString();
            map.put(key, value);
            i++;
        }
    } catch (FileNotFoundException e) {
        System.out.println("Could not open file: " + filePath);
        System.out.println("Make sure you are running from the project
root.");
        return;
    }

    long endTime = System.currentTimeMillis();
    System.out.println("Successfully inserted " + count + " elements
into HashMap.");
    System.out.println("Time taken: " + (endTime - startTime) + "
ms");
}
}

```

Input/output below:

```

/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -javaagent
Modified by: Aiden Wang

=== Part A: Manual HashMap Operations ===
Inserting keys: 13 21 5 37 15
Searched for 10 -> Not Found.
Searched for 21 -> Found! Value: 12

> Removing 20 and 37...
Searched for 37 -> Not Found.

> Map Entries via Iterator:
Key: 21 -> Value: 12
Key: 5 -> Value: 5
Key: 13 -> Value: 31
Key: 15 -> Value: 51

=== Part B: Inserting small1k.txt ===
Successfully inserted 1000 elements into HashMap.
Time taken: 25 ms

=== Part C: Inserting large100k.txt ===
Successfully inserted 100000 elements into HashMap.
Time taken: 104 ms

```

Exercise 2

Source code below:

```
package PA7;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
import net.datastructures.ChainHashMap;
import net.datastructures.Entry;

public class PA7_Ex2 {
    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        System.out.println("=== Part A: Manual ChainHashMap Operations  
===");
        runManualTests();

        System.out.println("\n=== Part B: Inserting small1k.txt ===");
        timeInsertion("PAs/PA7/Data/small1k.txt", 1000);

        System.out.println("\n=== Part C: Inserting large100k.txt ===");
        timeInsertion("PAs/PA7/Data/large100k.txt", 100000);
    }
    private static void runManualTests() {

        ChainHashMap<Integer, String> map = new ChainHashMap<>(11);

        int[] keys = {13, 21, 5, 37, 15};
        System.out.print("Inserting keys: ");
        for (int key : keys) {
            System.out.print(key + " ");
            String value = new
StringBuilder(String.valueOf(key)).reverse().toString();
            map.put(key, value);
        }
        System.out.println();

        searchAndPrint(map, 10);
        searchAndPrint(map, 21);
        System.out.println("\n> Removing 20 and 37...");
        map.remove(20);
        map.remove(37);

        searchAndPrint(map, 37);

        System.out.println("\n> Map Entries via Iterator:");
        for (Entry<Integer, String> entry : map.entrySet()) {
            System.out.println("Key: " + entry.getKey() + " -> Value: " +
entry.getValue());
        }
    }
    private static void searchAndPrint(ChainHashMap<Integer, String> map,
int key) {

        if (map.get(key) != null) {
```

```

        System.out.println("Searched for " + key + " -> Found! Value: " + map.get(key));
    } else {
        System.out.println("Searched for " + key + " -> Not Found.");
    }
}

private static void timeInsertion(String filePath, int count) {
    int capacity = (int) Math.ceil(count / 0.75);
    ChainHashMap<Integer, String> map = new ChainHashMap<>(capacity);

    long startTime = System.currentTimeMillis();

    try (Scanner scanner = new Scanner(new File(filePath))) {
        int i = 0;
        while (scanner.hasNextInt() && i < count) {
            int key = scanner.nextInt();
            String value = new
StringBuilder(String.valueOf(key)).reverse().toString();
            map.put(key, value);
            i++;
        }
    } catch (FileNotFoundException e) {
        System.out.println("Could not open file: " + filePath);
        System.out.println("Make sure you are running from the project
root.");
        return;
    }

    long endTime = System.currentTimeMillis();

    System.out.println("Successfully inserted " + count + " elements
into ChainHashMap.");
    System.out.println("Time taken: " + (endTime - startTime) + "
ms");
}
}

```

Input/output below:

```

/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Co
Modified by: Aiden Wang

=== Part A: Manual ChainHashMap Operations ===
Inserting keys: 13 21 5 37 15
Searched for 10 -> Not Found.
Searched for 21 -> Found! Value: 12

> Removing 20 and 37...
Searched for 37 -> Not Found.

> Map Entries via Iterator:
Key: 21 -> Value: 12
Key: 13 -> Value: 31
Key: 5 -> Value: 5
Key: 15 -> Value: 51

=== Part B: Inserting smallk.txt ===
Successfully inserted 1000 elements into ChainHashMap.
Time taken: 34 ms

=== Part C: Inserting large100k.txt ===
Successfully inserted 100000 elements into ChainHashMap.
Time taken: 301 ms

```

Exercise 3

Source code below:

```
package PA7;

import java.io.File;
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class PA7_Ex3 {
    public static void main(String[] args) throws Exception {
        System.out.println("Modified by: Aiden Wang\n");

        String path = "PAs/PA7/Data/USDeclIndFormatted.txt";
        Set<String> words = new HashSet<>();

        Scanner sc = new Scanner(new File(path));
        while (sc.hasNext()) {
            words.add(sc.next());
        }
        sc.close();
        System.out.println("Unique words: " + words.size());
        int[] aValues = {1, 37, 40, 41};

        for (int a : aValues) {
            Set<Integer> seen = new HashSet<>();
            int collisions = 0;

            for (String w : words) {
                int h = 0;
                for (int i = 0; i < w.length(); i++) {
                    h = h * a + w.charAt(i);
                }
                if (!seen.add(h)) {
                    collisions++;
                }
            }
            System.out.println("Parameter a = " + a + " -> Collisions: " + collisions);
        }
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/
Modified by: Aiden Wang

Unique words: 539
Parameter a = 1 -> Collisions: 178
Parameter a = 37 -> Collisions: 0
Parameter a = 40 -> Collisions: 0
Parameter a = 41 -> Collisions: 0

Process finished with exit code 0
```

Answer for Question 1

Yes. Hash table insertions with a good load factor take expected $O(1)$ time. Therefore, inserting N items takes $O(N)$ time overall (scaling linearly), meaning 100,000 items will naturally take about 100 times longer than 1,000 items.

Answer for Question 2

To map arbitrary/large integer hash codes into a valid array index range $[0, N-1]$ and minimize collisions. Common functions:

1. Division Method: $i \bmod N$.
2. MAD Method (Multiply-Add-and-Divide): $[(ai+b) \bmod p] \bmod N$.

Extra Credit –

Source code below:

```
package PA7;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.LinkedList;
import java.util.Random;
import java.util.Scanner;

public class PA7_EC2 {

    private static class SimpleChainHashMap {
        private LinkedList<Integer>[] table;
        private int capacity;
        private long totalProbes = 0;
        private int maxProbes = 0;
        private static final int PRIME = 109345121;
        private long scale, shift;

        @SuppressWarnings("unchecked")
        public SimpleChainHashMap(int cap, long scale, long shift) {
            this.capacity = cap;
            this.table = new LinkedList[cap];
            this.scale = scale;
            this.shift = shift;
        }

        public void put(int key) {
            int hash = (int) ((Math.abs(key * scale + shift) % PRIME) %
capacity);
            if (table[hash] == null) {
                table[hash] = new LinkedList<>();
            }
        }
    }
}
```

```
        int probes = 1 + table[hash].size();
        this.totalProbes += probes;
        if (probes > this.maxProbes) {
            this.maxProbes = probes;
        }
        table[hash].add(key);
    }

    public double getAverageProbes(int n) { return (double)
totalProbes / n; }
    public int getMaxProbes() { return maxProbes; }
}

public static void main(String[] args) {
    System.out.println("Modified by: Aiden Wang\n");
    String filePath = "PAs/PA7/Data/large100k.txt";
    int N = 100000;
    int[] data = new int[N];

    try (Scanner sc = new Scanner(new File(filePath))) {
        int i = 0;
        while (sc.hasNextInt() && i < N) {
            data[i++] = sc.nextInt();
        }
    } catch (FileNotFoundException e) {
        return;
    }

    Random rand = new Random(2024);
    long scale = rand.nextInt(109345121 - 1) + 1;
    long shift = rand.nextInt(109345121);
    double[] factors = {0.25, 0.5, 0.75, 0.9};

    for (double lf : factors) {
        int capacity = (int) Math.ceil(N / lf);
        SimpleChainHashMap map = new SimpleChainHashMap(capacity,
scale, shift);
        for (int key : data) {
            map.put(key);
        }

        System.out.println("Load Factor: " + lf);
        System.out.println("Table Size: " + capacity);
        System.out.printf("Average Probes: %.4f\n",
map.getAverageProbes(N));
        System.out.println("Max Probes: " + map.getMaxProbes());
        System.out.println();
    }
}
```

Input/output below:

```

/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -javaagent:/A
Modified by: Aiden Wang

Load Factor: 0.25
Table Size: 400000
Average Probes: 1.0000
Max Probes: 1

Load Factor: 0.5
Table Size: 200000
Average Probes: 1.0000
Max Probes: 1

Load Factor: 0.75
Table Size: 133334
Average Probes: 1.2710
Max Probes: 2

Load Factor: 0.9
Table Size: 111112
Average Probes: 1.2044
Max Probes: 2

```

Bug&Fix:

1. Problem: Snapshot Iterator Flaw

The `entrySet()` implementation utilizes a **Snapshot Iterator** design. It captures the state of the map at a single point in time by copying all entries into an `ArrayList` buffer, decoupling the iterator from live data.

```

ArrayList<Entry<K,V>> buffer = new ArrayList<>();
for (int h=0; h < capacity; h++)
    if (table[h] != null) buffer.add(table[h]);

```

2. Correction: Dynamic Inner Class Iterator

Replace the snapshot with a **Live Iterator** (inner class) that scans the `table` array in real-time using a dynamic cursor.

```

private class EntryIterator implements Iterator<Entry<K,V>> {
    private int j = -1;
    public Entry<K,V> next() {
        j++;
        while (j < capacity && isAvailable(j)) j++; // Skip null/defunct
slots
        return table[j];
    }
}

```